

Melhorias Aplicadas ao Planejamento de Testes

Durante a revisão e refinamento do planejamento de testes, foram implementadas as seguintes melhorias, visando aumentar a clareza, assertividade e eficiência na execução dos cenários:

✓ 1. Ajustes na Documentação dos Casos de Teste [↗](#)

- As descrições dos casos de teste **CT036**, **CT037** e **CT043** foram revisadas e reformuladas, com o objetivo de tornar os objetivos de cada teste mais claros e compreensíveis.
 - O caso de teste **CT053** foi **removido** por estar **duplicado**, evitando redundância e sobrecarga no plano de testes.
-

📌 2. Priorização das Rotas a Serem Testadas [↗](#)

Com base na documentação da API, nas **user stories**, no **fluxo da aplicação** e considerando o uso do **Robot Framework**, a priorização das rotas a serem testadas foi definida da seguinte forma:

1. **Rota** `/login`
 - Responsável pela autenticação e pela geração do token de acesso.
 - Etapa fundamental, pois habilita o uso de outras rotas protegidas.
 2. **Rota** `/usuarios`
 - Inclui funcionalidades de criação, atualização, consulta e exclusão de usuários.
 - Essencial para a gestão de contas e permissões.
 3. **Rota** `/produtos`
 - Envolve a administração dos produtos disponíveis para compra.
 - Impacta diretamente o fluxo de compra e operações do carrinho.
 4. **Rota** `/carrinhos`
 - Relacionada à finalização da experiência do usuário, como adição de produtos e conclusão da compra.
 - Executada após a validação das demais rotas dependentes.
-

⚖️ 3. Priorização de Casos de Teste com Base na Matriz de Risco [↗](#)

Dentro de cada rota, os **casos de teste foram organizados por nível de risco**, conforme a matriz de risco estabelecida anteriormente:

-  **Risco muito alto:** prioridade, com grande impacto para o projeto.
-  **Risco Alto:** casos com maior impacto e/ou alta probabilidade de falha.
-  **Risco Médio:** cenários com importância intermediária, impacto ou probabilidade moderados.
-  **Risco Baixo:** testes com baixa criticidade ou menor probabilidade de falha.

Essa abordagem permite mitigar os maiores riscos logo nas primeiras etapas da execução, aumentando a confiabilidade dos testes e da aplicação como um todo.